

第 11 章

图像金字塔

图像金字塔是由一幅图像的多个不同分辨率的子图所构成的图像集合。该组图像是由单个图像通过不断地降采样所产生的，最小的图像可能仅仅有一个像素点。图 11-1 是一个图像金字塔的示例。从图中可以看到，图像金字塔是一系列以金字塔形状排列的、自底向上分辨率逐渐降低的图像集合。

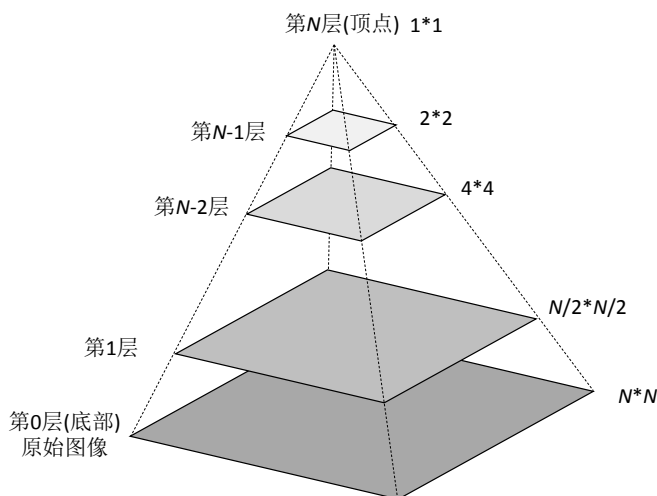


图 11-1 图像金字塔示例

通常情况下，图像金字塔的底部是待处理的高分辨率图像（原始图像），而顶部则为其低分辨率的近似图像。向金字塔的顶部移动时，图像的尺寸和分辨率都不断地降低。通常情况下，每向上移动一级，图像的宽和高都降低为原来的二分之一。

11.1 理论基础

图像金字塔是同一图像不同分辨率的子图集合，是通过对原图像不断地向下采样而产生的，即由高分辨率的图像（大尺寸）产生低分辨率的近似图像（小尺寸）。

最简单的图像金字塔可以通过不断地删除图像的偶数行和偶数列得到。例如，有一幅图像，其大小是 $N \times N$ ，删除其偶数行和偶数列后得到一幅 $(N/2) \times (N/2)$ 大小的图像。经过上述处理后，图像大小变为原来的四分之一，不断地重复该过程，就可以得到该图像的图像金字塔。

也可以先对原始图像滤波，得到原始图像的近似图像，然后将近似图像的偶数行和偶数列



删除以获取向下采样的结果。有多种滤波器可以选择。例如：

- **邻域滤波器**：采用邻域平均技术求原始图像的近似图像。该滤波器能够产生平均金字塔。
- **高斯滤波器**：采用高斯滤波器对原始图像进行滤波，得到高斯金字塔。这是 OpenCV 函数 `cv2.pyrDown()`所采用的方式。

高斯金字塔是通过不断地使用高斯金字塔滤波、采样所产生的，其过程如图 11-2 所示。



图 11-2 生成高斯金字塔的过程

经过上述处理后，原始图像与各次向下采样所得到的结果图像共同构成了高斯金字塔。例如，可以将原始图像称为第 0 层，第 1 次向下采样的结果图像称为第 1 层，第 2 次向下采样的结果图像称为第 3 层，以此类推。上述图像所构成的高斯金字塔如图 11-3 所示。在本章中为了便于表述，统一将图像金字塔中的底层称为第 0 层，底层上面的一层称为第 1 层，并以此类推。

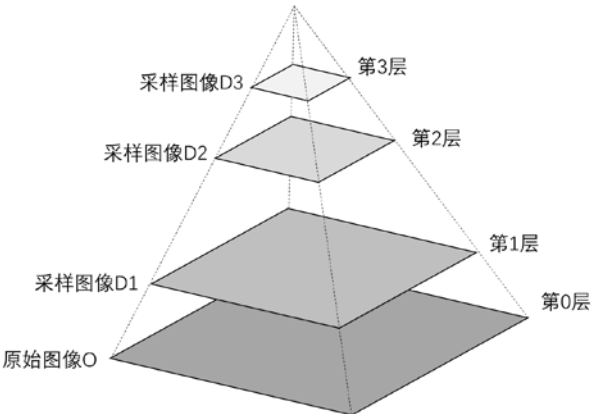


图 11-3 高斯金字塔

在向上采样的过程中，通常将图像的宽度和高度都变为原来的 2 倍。这意味着，向上采样的结果图像的大小是原始图像的 4 倍。因此，要在结果图像中补充大量的像素点。对新生成的像素点进行赋值，称为插值处理，该过程可以通过多种方式实现，例如最临近插值就是用最邻近的像素点给当前还没有值的像素点赋值。

有一种常见的向上采样，对像素点以补零的方式完成插值。通常是在每列像素点的右侧插入值为零的列，在每行像素点的下方插入值为零的行。在图 11-4 中，左侧是要进行向上采样的

4 个像素点，右侧是向上采样时进行补零后的处理结果。



图 11-4 向上采样示例

接下来，使用向下采样时所用的高斯滤波器（高斯核）对补零后的图像进行滤波处理，以获取向上采样的结果图像。但是需要注意，此时图像中四分之三像素点的值都是零。所以，要将高斯滤波器系数乘以 4，以保证得到的像素值范围在其原有像素值范围内。

例如，针对图 11-4 右侧的像素点，其对应的是 8 位图像，像素值的范围是[0, 255]。由于其中四分之三的像素点的值都为零，如果直接使用高斯滤波器对其进行卷积计算，会导致像素值的范围变为[0, 255*1/4]。所以，要将所使用的高斯滤波器系数乘以 4，以保证得到像素值的范围仍旧在[0, 255]内。

或者，从另一个角度理解，在原始图像内每个像素点的右侧列插入零值列，在每个像素点的下一行插入零值行，将图像变为原来的两倍宽、两倍高。接下来，将补零后的图像用向下采样时所用的高斯滤波器进行卷积运算。最后，将图像内每个像素点的值乘以 4，以保证像素值的范围与原始图像的一致。

通过以上分析可知，向上采样和向下采样是相反的两种操作。但是，由于向下采样会丢失像素值，所以这两种操作并不是可逆的。也就是说，对一幅图像先向上采样、再向下采样，是无法恢复其原始状态的；同样，对一幅图像先向下采样、再向上采样也无法恢复到原始状态。

11.2 pyrDown 函数及使用

OpenCV 提供了函数 `cv2.pyrDown()`，用于实现图像高斯金字塔操作中的向下采样，其语法形式为：

```
dst = cv2.pyrDown( src[, dstsize[, borderType]] )
```

其中：

- `dst` 为目标图像。
- `src` 为原始图像。
- `dstsize` 为目标图像的大小。
- `borderType` 为边界类型，默认值为 `BORDER_DEFAULT`，且这里仅支持 `BORDER_DEFAULT`。

默认情况下，输出图像的大小为 $\text{Size}((\text{src.cols}+1)/2, (\text{src.rows}+1)/2)$ 。在任何情况下，图像尺寸必须满足如下条件：

$$|\text{dst.width} * 2 - \text{src.cols}| \leq 2$$

$$|\text{dst.height} * 2 - \text{src.rows}| \leq 2$$

`cv2.pyrDown()` 函数首先对原始图像进行高斯滤波变换，以获取原始图像的近似图像。比如，高斯滤波变换所使用的核（高斯滤波器）为：



$$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

在获取近似图像后，该函数通过抛弃偶数行和偶数列来实现向下采样。

【例 11.1】 使用函数 `cv2.pyrDown()` 对一幅图像进行向下采样，观察采样的结果。

根据题目要求，编写程序如下：

```
import cv2
o=cv2.imread("lena.bmp",cv2.IMREAD_GRAYSCALE)
r1=cv2.pyrDown(o)
r2=cv2.pyrDown(r1)
r3=cv2.pyrDown(r2)
print("o.shape=",o.shape)
print("r1.shape=",r1.shape)
print("r2.shape=",r2.shape)
print("r3.shape=",r3.shape)
cv2.imshow("original",o)
cv2.imshow("r1",r1)
cv2.imshow("r2",r2)
cv2.imshow("r3",r3)
cv2.waitKey()
cv2.destroyAllWindows()
```

本例使用 `cv2.pyrDown()` 函数进行了 3 次向下采样，并且用 `print()` 函数输出了每次采样结果图像的大小。`cv2.imshow()` 函数显示了原始图像和经过 3 次向下采样后得到的结果图像。

程序运行后，会显示如下结果：

```
o.shape= (512, 512)
r1.shape= (256, 256)
r2.shape= (128, 128)
r3.shape= (64, 64)
```

从上述结果可知，经过向下采样后，图像的行和列的数量都会变为原来的二分之一，图像整体的大小会变为原来的四分之一。

程序还会显示如图 11-5 所示图像，图像的大小就是上述输出结果所显示的大小。这里为了便于比较，将它们调整成了等高格式展示。读者通过标题栏与图像的比例关系，可以推断出各个图像的大致尺寸比例。其中：

- (a)图是原始图像 o。
- (b)图是对原始图像 o 进行向下采样后的结果图像 r1。
- (c)图是对图像 r1 进行向下采样后的结果图像 r2。
- (d)图是对图像 r2 进行向下采样后的结果图像 r3。

可以看到，经过向下采样后，图像的分辨率会变低。



图 11-5 【例 11.1】程序的运行结果

11.3 pyrUp 函数及使用

在 OpenCV 中,使用函数 `cv2.pyrUp()`实现图像金字塔操作中的向上采样,其语法形式如下:

```
dst = cv2.pyrUp( src[, dstsize[, borderType]] )
```

其中:

- `dst` 为目标图像。
- `src` 为原始图像。
- `dstsize` 为目标图像的大小。
- `borderType` 为边界类型,默认值为 `BORDER_DEFAULT`,且这里仅支持 `BORDER_DEFAULT`。

默认情况下,目标图像的大小为 `Size(src.cols*2, src.rows*2)`。在任何情况下,图像尺寸需要满足下列条件:

$$|\text{dst.width} - \text{src.cols} * 2| \leq \text{mod}(\text{dst.width}, 2)$$

$$|\text{dst.height} - \text{src.rows} * 2| \leq \text{mod}(\text{dst.height}, 2)$$

对图像向上采样时,在每个像素的右侧、下方分别插入零值列和零值行,得到一个偶数行、偶数列(即新增的行、列)都是零值的新图像 `New`。接下来,用向下采样时所需要的高斯滤波器对新图像 `New` 进行滤波,得到向上采样的结果图像。需要注意的是,为了确保像素值区间在向上采样后与原始图像保持一致,需要将高斯滤波器的系数乘以 4。

上一段描述的是 OpenCV 函数 `cv2.pyrUp()`所实现的向上采样过程。了解上述过程,有助于我们更好地理解和使用该函数。但是,OpenCV 库的目的就是要让我们忽略这些细节,直接使用函数 `cv2.pyrUp()`完成向上采样。所以,在刚开始的学习阶段,我们也可以先忽略这些细节。

【例 11.2】使用函数 `cv2.pyrUp()`对一幅图像进行向上采样,观察采样的结果。

根据题目要求,编写程序如下:

```
import cv2
o=cv2.imread("lenas.bmp")
r1=cv2.pyrUp(o)
r2=cv2.pyrUp(r1)
r3=cv2.pyrUp(r2)
print("o.shape=",o.shape)
```

```

print("r1.shape=", r1.shape)
print("r2.shape=", r2.shape)
print("r3.shape=", r3.shape)
cv2.imshow("original", o)
cv2.imshow("r1", r1)
cv2.imshow("r2", r2)
cv2.imshow("r3", r3)
cv2.waitKey()
cv2.destroyAllWindows()

```

本例使用 `cv2.pyrUp()` 函数对图像进行了 3 次向上采样。采样后，利用 `print()` 函数输出了每次采样结果图像的大小，用 `cv2.imshow()` 函数显示了原始图像和 3 次向上采样后的结果图像。

程序运行后，会输出如下结果：

```

o.shape= (64, 64, 3)
r1.shape= (128, 128, 3)
r2.shape= (256, 256, 3)
r3.shape= (512, 512, 3)

```

从上述输出结果可知，经过向上采样后，图像的宽度和高度都会变为原来的 2 倍，图像整体大小会变为原来的 4 倍。

程序还会显示如图 11-6 所示图像，图像大小就是上述输出结果所显示的大小。这里为了便于对比，将它们调整成了等高格式展示。读者可以通过各个图像内标题栏所占比例，看出各个图像大致的尺寸关系。

在图 11-6 中：

- (a)图是原始图像 o。
- (b)图是对原始图像 o 进行向上采样后的结果图像 r1。
- (c)图是对图像 r1 进行向上采样后的结果图像 r2。
- (d)图是对图像 r2 进行向上采样后的结果图像 r3。

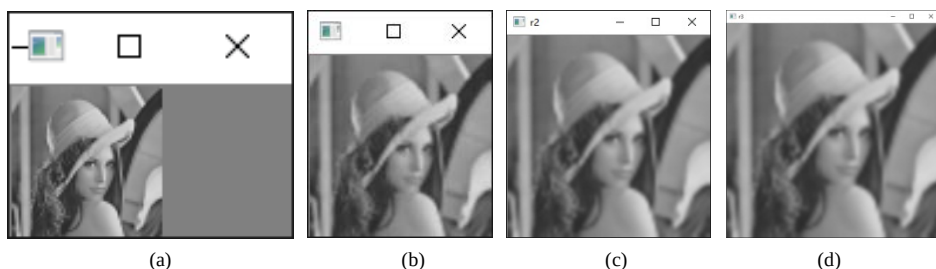


图 11-6 【例 11.2】程序的运行结果

11.4 采样可逆性的研究

图像在向上采样后，整体尺寸变为原来的 4 倍；在向下采样后，整体尺寸变为原来的四分之一。图 11-7 展示了图像在采样前后的大小变化关系。一幅 $M \times N$ 大小的图像经过向下采样后大小会变为 $(M/2) \times (N/2)$ ；一幅 $M \times N$ 大小的图像经过向上采样后大小会变为 $(2M) \times (2N)$ 。

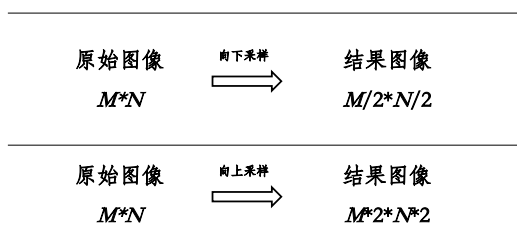


图 11-7 采样前后图像大小的变化

一幅图像在先后经过向下采样和向上采样后，会恢复为原始大小，如图 11-8 所示。

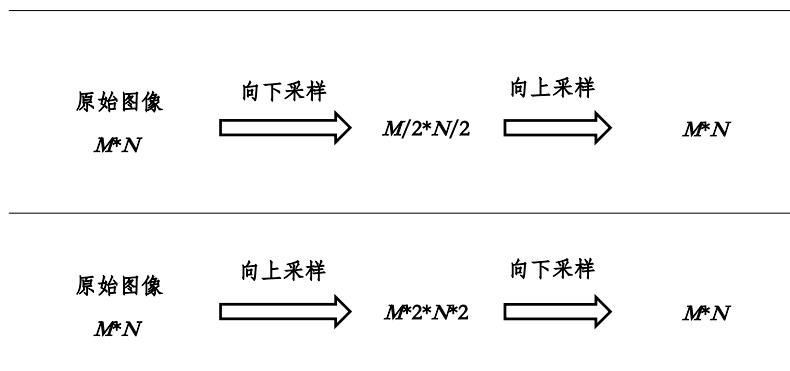


图 11-8 图像先后经过向下采样和向上采样后，会恢复为原始大小

虽然一幅图像在先后经过向下采样、向上采样后，会恢复为原始大小，但是向上采样和向下采样不是互逆的。也就是说，虽然在经历两次采样操作后，得到的结果图像与原始图像的大小一致，肉眼看起来也相似，但是二者的像素值并不是一致的。

【例 11.3】使用函数 `cv2.pyrDown()` 和 `cv2.pyrUp()`，先后将一幅图像进行向下采样、向上采样，观察采样的结果及结果图像与原始图像的差异。

根据题目要求，编写程序如下：

```
import cv2
o=cv2.imread("lena.bmp")
down=cv2.pyrDown(o)
up=cv2.pyrUp(down)
diff=up-o #构造 diff 图像，查看 up 与 o 的区别
print("o.shape=",o.shape)
print("up.shape=",up.shape)
cv2.imshow("original",o)
cv2.imshow("up",up)
cv2.imshow("difference",diff)
cv2.waitKey()
cv2.destroyAllWindows()
```

本例在尝试向大家说明，原始图像先后经过向下采样、向上采样后，所得到的结果图像与原始图像的大小一致，看起来也很相似，但是它们的像素值并不是一致的。



运行程序，会显示如下输出：

```
o.shape= (512, 512, 3)
up.shape= (512, 512, 3)
```

从上述输出结果可以得知，图像在先后经过向下采样、向上采样后，得到的结果图像与原始图像大小一致。

同时，程序还会显示如图 11-9 所示图像。其中：

- (a)图是原始图像 o。
- (b)图是对图像 down（通过对原始图像 o 向下采样得到）进行向上采样后获得的结果图像 up。
- (c)图是对图像 up 和原始图像 o 进行减法运算的结果（差值）图像 diff。图像 diff 反映的是图像 up 和原始图像 o 的差值。



图 11-9 【例 11.3】程序的运行结果

从本例的结果可知，原始图像 o 与先后经过向下采样、向上采样得到的结果图像 up，在大小上是相等的，在外观上是相似的，但是它们的像素值并不一致。

【例 11.4】使用函数 `cv2.pyrUp()` 和 `cv2.pyrDown()`，先后将一幅图像进行向上采样、向下采样，观察采样的结果及结果图像与原始图像的差异。

根据题目要求，编写程序如下：

```
import cv2
o=cv2.imread("lena.bmp")
up=cv2.pyrUp(o)
down=cv2.pyrDown(up)
diff=down-o #构造 diff 图像，查看 down 与 o 的区别
print("o.shape=",o.shape)
print("down.shape=",down.shape)
cv2.imshow("original",o)
cv2.imshow("down",down)
cv2.imshow("difference",diff)
cv2.waitKey()
cv2.destroyAllWindows()
```

本例尝试向大家说明，一幅图像在先后经过向上采样、向下采样后，所得到的结果图像大

小与原始图像一致，看起来也很相似，但是它们的像素值并不是一致的。

运行程序，会显示如下输出：

```
o.shape= (512, 512, 3)
down.shape= (512, 512, 3)
```

从上述输出可得知，图像在先后经过向上采样、向下采样后，得到的结果图像与原始图像大小一致。

同时，程序还会显示如图 11-10 所示图像。其中：

- (a)图是原始图像 o。
- (c)图是对图像 up(通过对原始图像 o 向上采样得到)进行向下采样后的结果图像 down。
- (d)图是对图像 down 和原始图像 o 进行减法运算的结果图像 diff。图像 diff 反映的是图像 down 和原始图像 o 的差值。



图 11-10 【例 11.4】程序的运行结果

从本例的结果可知，原始图像 o 与先后经过向上采样、向下采样得到的图像 down，在大小上是相等的，在外观上是相似的，但是它们的像素值并不一致。

11.5 拉普拉斯金字塔

前面我们介绍了高斯金字塔，高斯金字塔是通过对一幅图像一系列的向下采样所产生的。有时，我们希望通过金字塔中的小图像进行向上采样以获取完整的大尺寸高分辨率图像，这时就需要用到拉普拉斯金字塔。

11.5.1 定义

前面我们已经介绍过，一幅图像在经过向下采样后，再对其进行向上采样，是无法恢复为原始状态的。对此，我们也用程序进行了验证。向上采样并不是向下采样的逆运算。这是很明显的，因为向下采样时在使用高斯滤波器处理后还要抛弃偶数行和偶数列，不可避免地要丢失一些信息。

为了在向上采样时能够恢复具有较高分辨率的原始图像，就要获取在采样过程中所丢失的信息，这些丢失的信息就构成了拉普拉斯金字塔。



拉普拉斯金字塔的定义形式为：

$$L_i = G_i - \text{pyrUp}(G_{i+1})$$

式中：

- L_i 表示拉普拉斯金字塔中的第 i 层。
- G_i 表示高斯金字塔中的第 i 层。

拉普拉斯金字塔中的第 i 层，等于“高斯金字塔中的第 i 层”与“高斯金字塔中的第 $i+1$ 层的向上采样结果”之差。图 11-11 展示了高斯金字塔和拉普拉斯金字塔的对应关系。

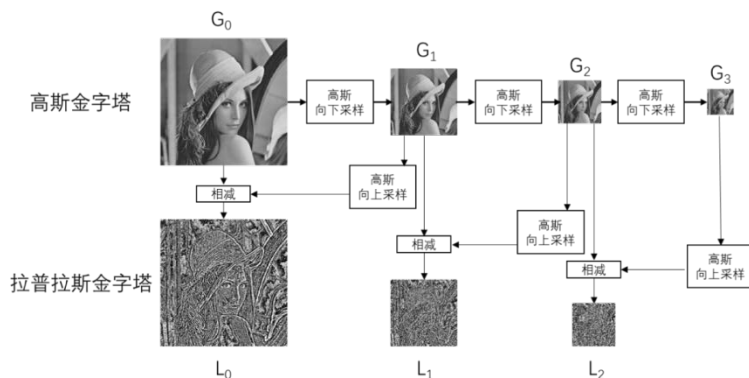


图 11-11 拉普拉斯金字塔

【例 11.5】 使用函数 `cv2.pyrDown()` 和 `cv2.pyrUp()` 构造拉普拉斯金字塔。

根据题目要求，编写程序如下：

```
import cv2
0=cv2.imread("lena.bmp")
G0=0
G1=cv2.pyrDown(G0)
G2=cv2.pyrDown(G1)
G3=cv2.pyrDown(G2)
L0=G0-cv2.pyrUp(G1)
L1=G1-cv2.pyrUp(G2)
L2=G2-cv2.pyrUp(G3)
print("L0.shape=",L0.shape)
print("L1.shape=",L1.shape)
print("L2.shape=",L2.shape)
cv2.imshow("L0",L0)
cv2.imshow("L1",L1)
cv2.imshow("L2",L2)
cv2.waitKey()
cv2.destroyAllWindows()
```

程序运行后，会输出如下运行结果：

```
L0.shape= (512, 512, 3)
L1.shape= (256, 256, 3)
```

$L2.shape = (128, 128, 3)$

程序还会显示如图 11-12 所示图像。这里为了便于对比，将它们调整成了等高格式展示。读者可以通过图中标题栏的大小，得知原图的大小比例关系。

在图 11-12 中：

- (a)图是通过语句“ $L0=G0-cv2.pyrUp(G1)$ ”，用“原始图像 $G0$ ”减去“图像 $G1$ 的向上采样结果”，得到的拉普拉斯金字塔的第 0 层 $L0$ 。
- (b)图是通过语句“ $L1=G1-cv2.pyrUp(G2)$ ”，用“图像 $G1$ ”减去“图像 $G2$ 的向上采样结果”，得到的拉普拉斯金字塔的第 1 层 $L1$ 。
- (c)图是通过语句“ $L2=G2-cv2.pyrUp(G3)$ ”，用“图像 $G2$ ”减去“图像 $G3$ 的向上采样结果”，得到的拉普拉斯金字塔的第 2 层 $L2$ 。

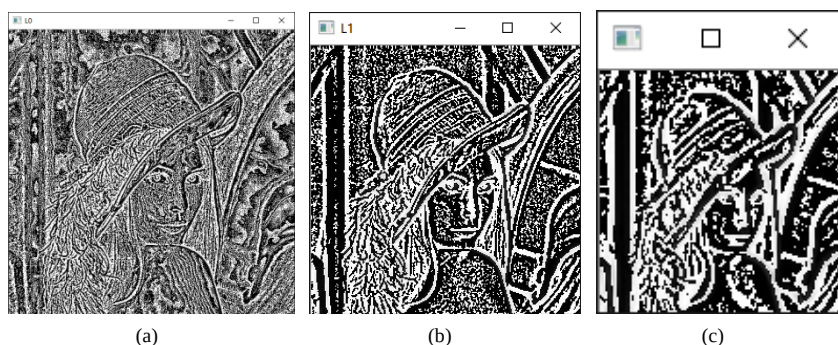


图 11-12 拉普拉斯金字塔各层的图像

11.5.2 应用

拉普拉斯金字塔的作用在于，能够恢复高分辨率的图像。图 11-13 演示了如何通过拉普拉斯金字塔恢复高分辨率图像。其中，右图是对左图的简化。

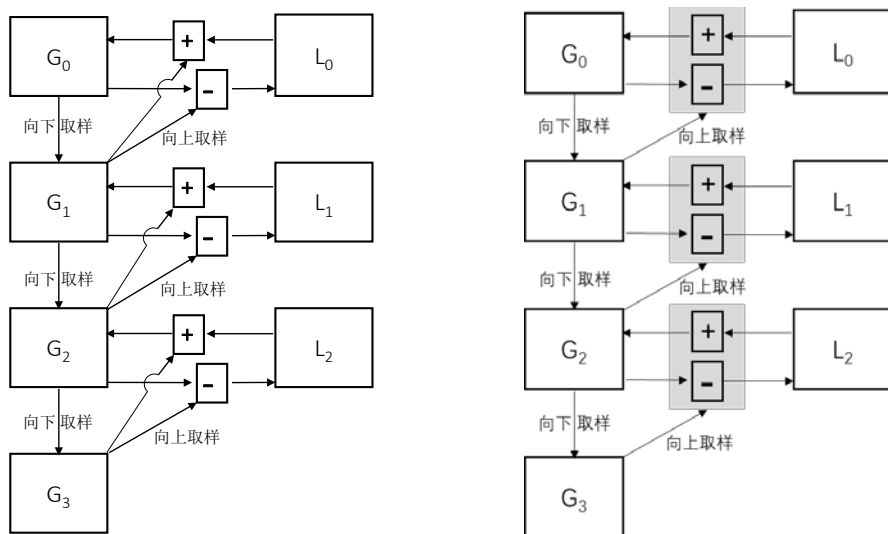


图 11-13 利用拉普拉斯金字塔恢复高分辨率图像

图 11-13 中的各个标记的含义如下：

- G_0 、 G_1 、 G_2 、 G_3 分别是高斯金字塔的第 0 层、第 1 层、第 2 层、第 3 层。
- L_0 、 L_1 、 L_2 分别是拉普拉斯金字塔的第 0 层、第 1 层、第 2 层。
- 向下的箭头表示向下采样操作（对应 `cv2.pyrDown()` 函数）。
- 向右上方的箭头表示向上采样操作（对应 `cv2.pyrUp()` 函数）。
- 加号 “+” 表示加法操作。
- 减号 “-” 表示减法操作。

在图 11-13 中，描述的操作及关系有：

- 向下采样（高斯金字塔的构成）

```
G1=cv2.pyrDown(G0)
```

```
G2=cv2.pyrDown(G1)
```

```
G3=cv2.pyrDown(G2)
```

- 拉普拉斯金字塔

```
L0=G0-cv2.pyrUp(G1)
```

```
L1=G1-cv2.pyrUp(G2)
```

```
L2=G2-cv2.pyrUp(G3)
```

- 向上采样恢复高分辨率图像

```
G0=L0+cv2.pyrUp(G1)
```

```
G1=L1+cv2.pyrUp(G2)
```

```
G2=L2+cv2.pyrUp(G3)
```

上述关系是通过数学运算推导得到的。例如，已知 $L_0=G_0-cv2.pyrUp(G_1)$ ，将表达式右侧的 `cv2.pyrUp(G1)` 移到左侧，就得到了表达式 $G_0=L_0+cv2.pyrUp(G_1)$ 。除此之外， G_1 和 G_2 都可以通过拉普拉斯金字塔的构造表达式得到。在前面已经介绍过了，构造拉普拉斯金字塔的目的就是为了恢复高分辨率的图像。

【例 11.6】 编写程序，使用拉普拉斯金字塔及高斯金字塔恢复原始图像。

根据题目要求，编写程序如下：

```
1. import cv2
2. import numpy as np
3. 0=cv2.imread("lena.bmp")
4. G0=0
5. G1=cv2.pyrDown(G0)
6. L0=0-cv2.pyrUp(G1)
7. R0=L0+cv2.pyrUp(G1) # 通过拉普拉斯图像复原的原始图像
8. print("0.shape=",0.shape)
9. print("R0.shape=",R0.shape)
10. result=R0-0 # 将 0 和 R0 做减法运算
11. # 计算 result 的绝对值，避免求和时负负为正，3+(-3)=0
12. result=abs(result)
13. # 计算 result 所有元素的和
14. print("原始图像 0 与恢复图像 R0 之差的绝对值和: ",np.sum(result))
```

本程序中：

- 第 4 行将原始图像 O 赋值给 G0，对原始图像重命名，方便后续进行观察和说明。
- 第 5 行对图像 G0（高斯金字塔的第 0 层）进行向下采样，得到向下采样的结果，即高斯金字塔的第 1 层 G1。
- 第 6 行使用原始图像 O 减去图像 G1 的向上采样，得到拉普拉斯金字塔的第 0 层 L0。
- 第 7 行使用 L0 加上图像 G1 的向上采样结果，得到恢复的原始图像 RO。
- 第 10 行计算原始图像 O 与恢复的图像 RO 之差。如果差值图像 result 的元素值全部为 0，则说明图像 RO 和图像 O 内的像素值完全一致；否则，不一致。
- 第 12 行计算 result 内所有元素值的绝对值，然后计算 result 内所有值的和是否为零，以此判断其所有元素值是否都为零。这样写是为了避免 result 内的正数值和与负数值之和正好抵消的情况。比如，result 中存在两个不为零的值，一个为正值“3”，另一个为负值“-3”，虽然并非所有元素都为零，但是它们相加求和的结果是零。在这种情况下，通过和为零来判断所有元素都是零，显然是不正确的。而取绝对值后再求和，通过绝对值是否为零来判断是否所有元素都是零，则避免了可能的误判情况。

程序运行后，会输出如下运行结果：

```
O.shape= (512, 512, 3)
```

```
RO.shape= (512, 512, 3)
```

原始图像 O 与恢复的图像 RO 之差的绝对值和：0

从程序运行结果可以看到，原始图像与恢复图像差值的绝对值和为“0”。这说明使用拉普拉斯金字塔恢复的图像与原始图像完全一致。

【例 11.7】编写程序，使用拉普拉斯金字塔及高斯金字塔恢复高斯金字塔内的多层图像。

根据题目要求，编写程序如下：

```
1. import cv2
2. import numpy as np
3. O=cv2.imread("lena.bmp")
4. #=====生成高斯金字塔=====
5. G0=O
6. G1=cv2.pyrDown(G0)
7. G2=cv2.pyrDown(G1)
8. G3=cv2.pyrDown(G2)
9. #=====生成拉普拉斯金字塔=====
10. L0=G0-cv2.pyrUp(G1) #拉普拉斯金字塔第 0 层
11. L1=G1-cv2.pyrUp(G2) #拉普拉斯金字塔第 1 层
12. L2=G2-cv2.pyrUp(G3) #拉普拉斯金字塔第 2 层
13. #=====复原 G0=====
14. RG0=L0+cv2.pyrUp(G1) #通过拉普拉斯图像复原的原始图像 G0
15. print("G0.shape=", G0.shape)
16. print("RG0.shape=", RG0.shape)
17. result=RG0-G0 #将 RG0 和 G0 相减
18. #计算 result 的绝对值，避免求和时负负为正，3+(-3)=0
19. result=abs(result)
20. #计算 result 所有元素的和
```



```

21. print("原始图像 G0 与恢复图像 RG0 差值的绝对值和: ", np.sum(result))
22. #=====复原 G1=====
23. RG1=L1+cv2.pyrUp(G2) #通过拉普拉斯图像复原 G1
24. print("G1.shape=", G1.shape)
25. print("RG1.shape=", RG1.shape)
26. result=RG1-G1 #将 RG1 和 G1 相减
27. print("原始图像 G1 与恢复图像 RG1 之差的绝对值和: ", np.sum(abs(result)))
28. #=====复原 G2=====
29. RG2=L2+cv2.pyrUp(G3) #通过拉普拉斯图像复原 G2
30. print("G2.shape=", G2.shape)
31. print("RG2.shape=", RG2.shape)
32. result=RG2-G2 #将 RG2 和 G2 相减
33. print("原始图像 G2 与恢复图像 RG2 之差的绝对值和: ", np.sum(abs(result)))

```

本程序中：

- 第 14 行通过“拉普拉斯金字塔的第 0 层 L0”与“高斯金字塔第 1 层 G1 向上采样的结果 cv2.pyrUp(G1)”之和，恢复了原始图像 G0，得到 RG0。
- 第 23 行通过“拉普拉斯金字塔的第 1 层 L1”与“高斯金字塔第 2 层 G2 向上采样的结果 cv2.pyrUp(G2)”之和，恢复了原始图像 G1，得到 RG1。
- 第 29 行通过“拉普拉斯金字塔的第 2 层 L2”与“高斯金字塔第 3 层 G3 向上采样的结果 cv2.pyrUp(G3)”之和，恢复了原始图像 G2，得到 RG2。

程序运行后，会输出如下运行结果：

```

G0.shape= (512, 512, 3)
RG0.shape= (512, 512, 3)
原始图像 G0 与恢复图像 RG0 之差的绝对值和: 0
G1.shape= (256, 256, 3)
RG1.shape= (256, 256, 3)

原始图像 G1 与恢复图像 RG1 之差的绝对值和: 0

G2.shape= (128, 128, 3)
RG2.shape= (128, 128, 3)

原始图像 G2 与恢复图像 RG2 之差的绝对值和: 0

```

从程序运行结果可以看到，在每一层中，原始图像与恢复图像差值的绝对值和都为“0”。这说明使用拉普拉斯金字塔恢复的图像与原始图像完全一致。